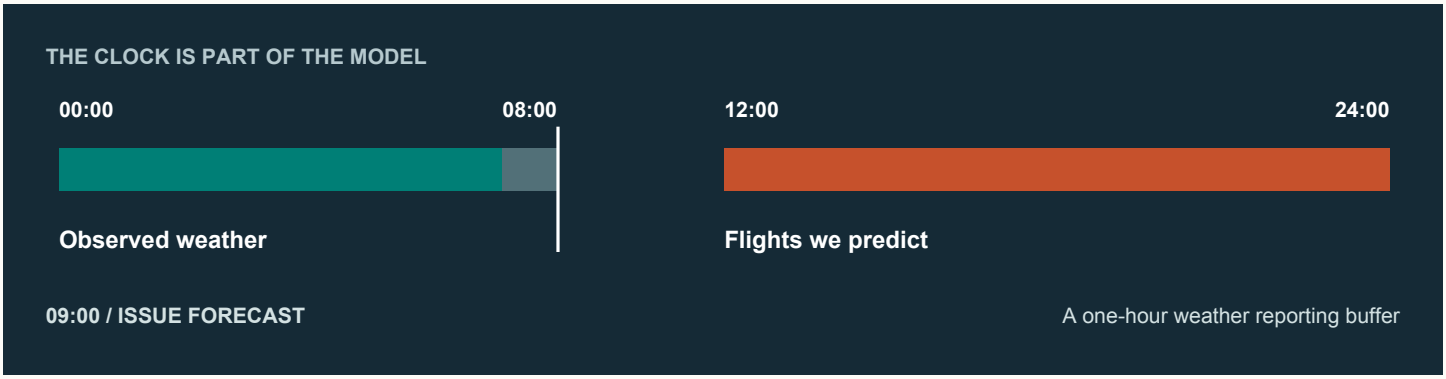
AN AIRPORT-DAY FORECAST, BUILT BEFORE LUNCH

The 9 a.m. forecast

Can morning weather help airport planners anticipate afternoon departure delays? We combine flight schedules with weather recorded before 8 a.m., predict the mean delay of flights scheduled from noon onward, and test the model on a later month. Weather cuts validation error by **12.3%** in 2023. In January 2024, that advantage disappears. A few severe days overwhelm the forecast.



Data

We combine **BTS flight records** with **Iowa Environmental Mesonet weather observations**. The 2023 flights come through nycflights23; January 2024 comes directly from BTS. Both use the same reporting-carrier definitions and contain the same 14 carriers. We aggregate each source by airport and local date, then check a one-to-one join. The panel contains **1,188 airport-days**: 1,095 for development and 93 for the future test.

The outcome includes early departures and averages only recorded delays. Of 262,528 scheduled afternoon flights in development, **8,180 lack a delay outcome**; they remain in schedule counts. Development mean delay is **21.1 minutes**, versus a median of **12.8**. Weather is available for every airport-day; missing individual readings use training-only medians. Trace rain remains a small positive amount. No afternoon weather enters the forecast.

1,095
DEVELOPMENT AIRPORT-DAYS

93
FUTURE AIRPORT-DAYS

08:00
WEATHER CUTOFF

Schedules are reconstructed from final records, so historical schedule revisions and feed latency remain untested. Missing delay outcomes, including cancellations, are a separate operational risk. January supplies 7.8% of the combined panel.

Sources: BTS / nycflights23 / Iowa Environmental Mesonet

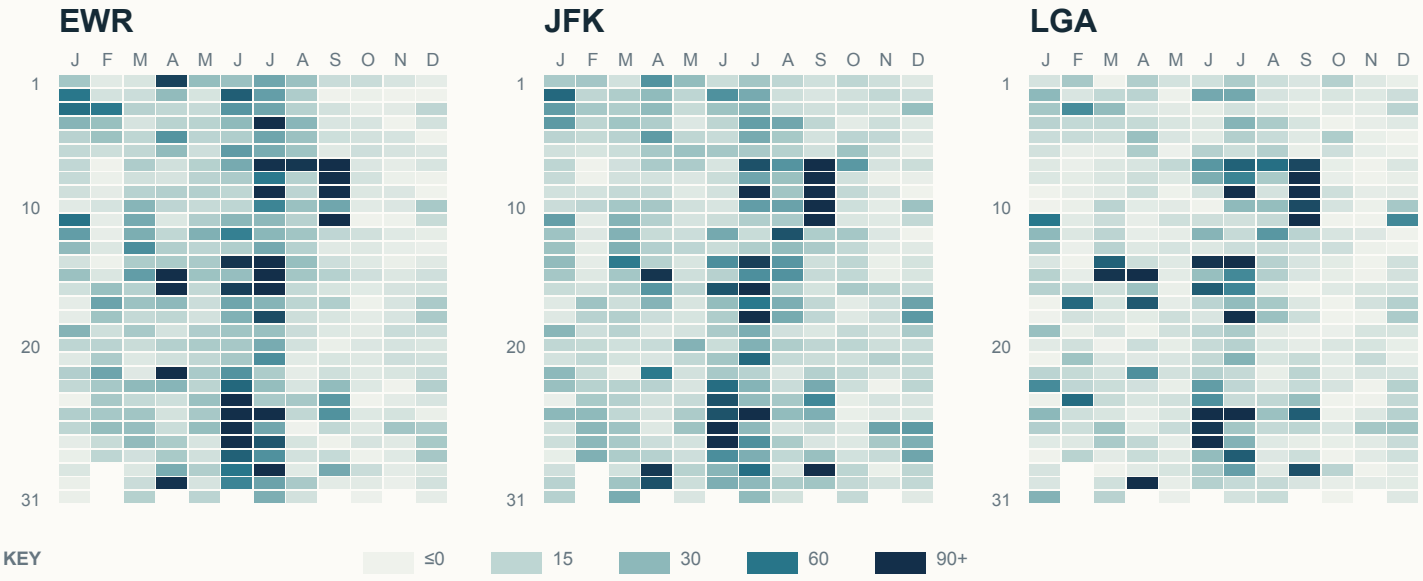
01 / SEASON AND WEATHER

Bad mornings are not the whole story.

Delay clusters in summer, including days without low visibility or strong morning wind. Weather matters differently across airports and seasons.

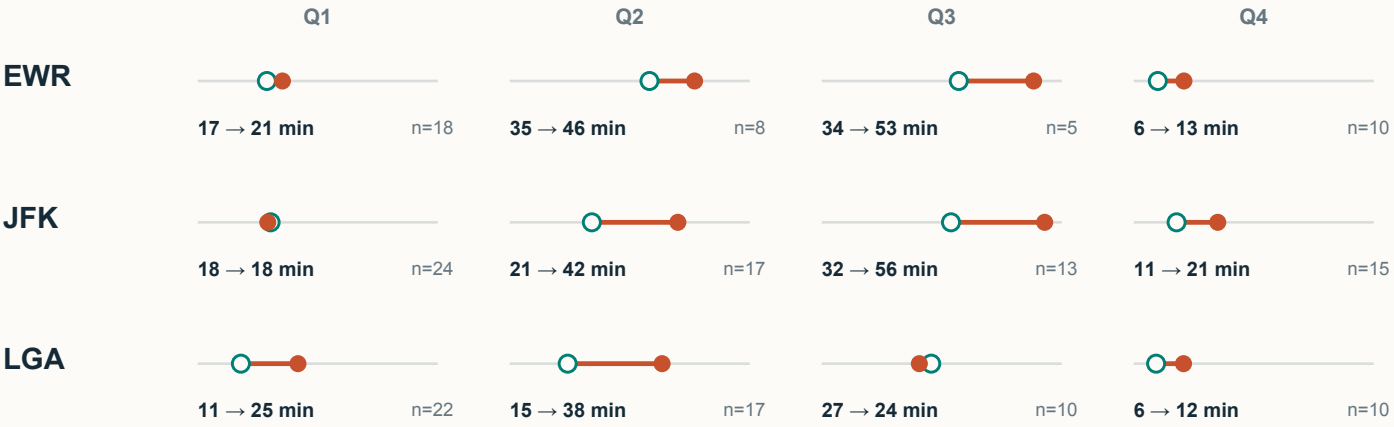
Every afternoon in 2023

Mean departure delay, minutes



Within the same airport and quarter

Mean afternoon delay: other mornings ○ → low visibility / high wind ●



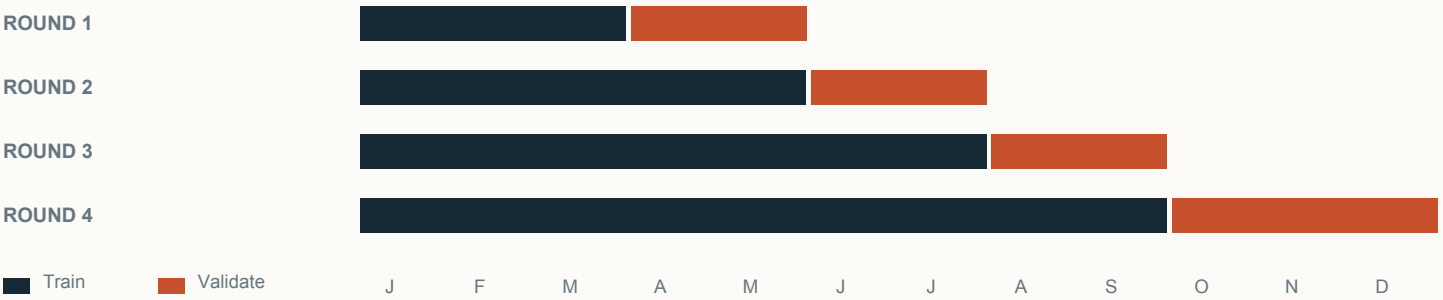
Weather flag: minimum pre-8 a.m. visibility below 3 miles or maximum wind at least 20 knots. Counts show flagged mornings; some cells are small. Mean delay is higher in 10 of 12 comparisons, not all 12. These are descriptive associations, not causal estimates.

02 / PAIRED FORWARD COMPARISONS

Give weather a fair test.

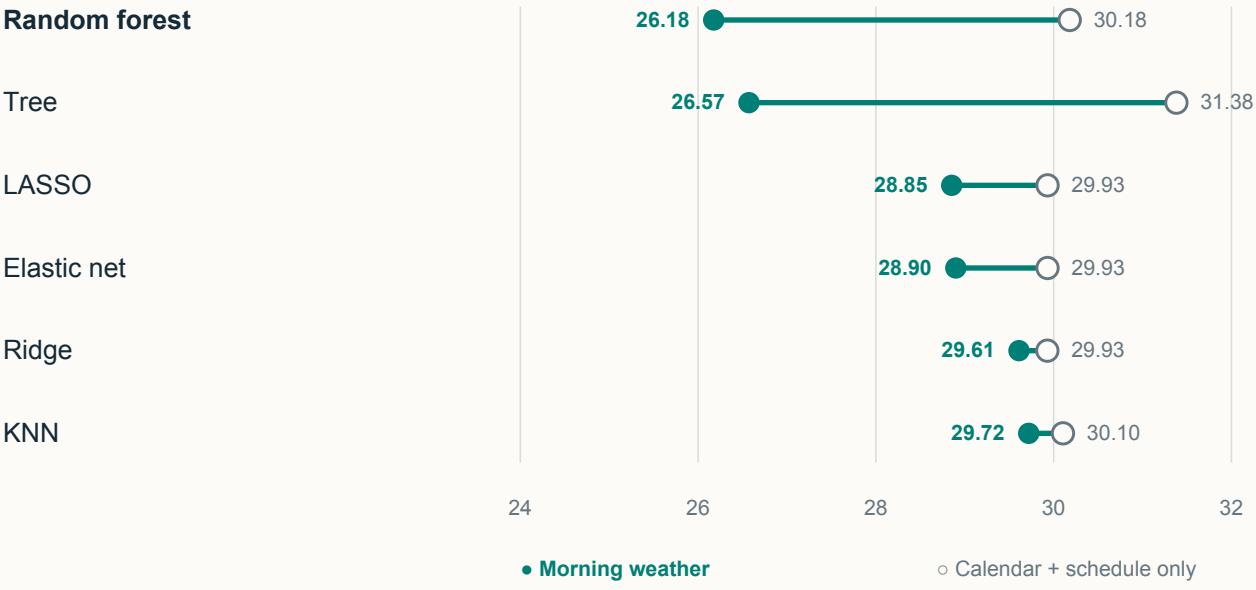
Tune each method twice: once with calendar and schedule inputs, then with those inputs plus morning weather. Both versions predict the same later dates.

The validation clock only moves forward



Weather helps more than changing the algorithm.

Pooled forward-validation RMSE, minutes • lower is better



The strongest no-weather choice is the airport average: **29.84 minutes RMSE**. The weather forest lowers that to **26.18**, or **12.3%**. But the gain varies by season:

APR-MAY	JUN-JUL	AUG-SEP	OCT-DEC
24.8 → 23.4	44.9 → 34.0	28.7 → 33.1	19.2 → 13.8

Airport-average → morning-weather RMSE, minutes. Each window is later than its training data.

03 / SELECT BEFORE TESTING

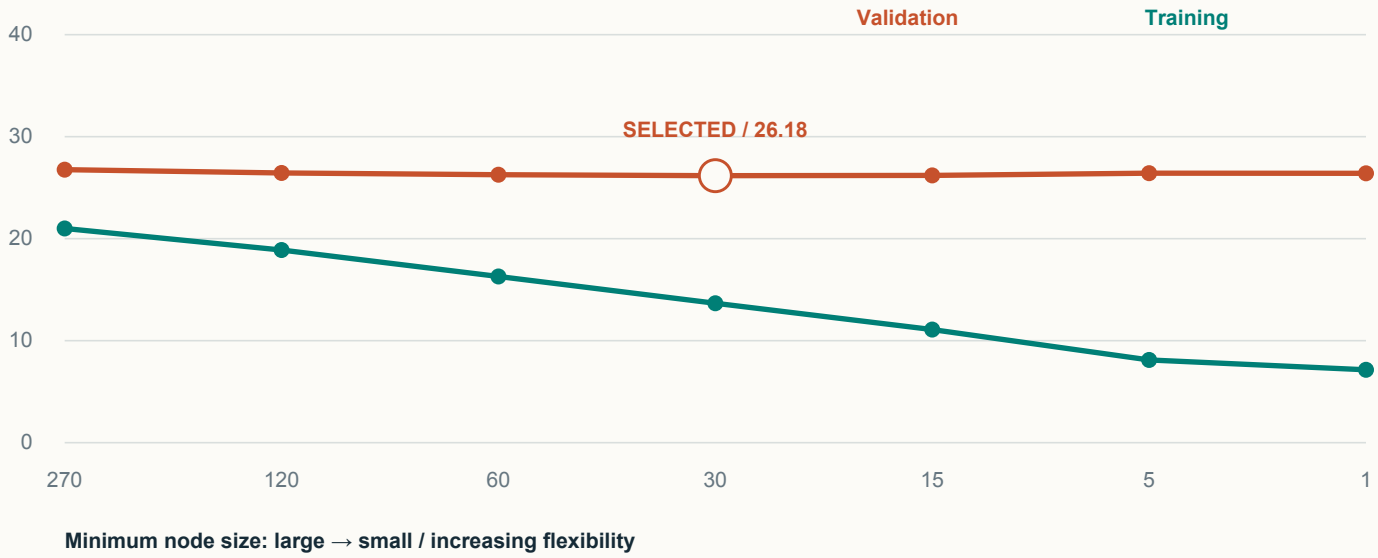
A restrained forest wins.

The selected model averages 500 trees, considers all 22 predictors at a split, and stops splitting nodes smaller than 30 observations. All tuning uses 2023 only.

Inputs cover airport, weekday, annual seasonality, afternoon schedule size, route distance, long-haul and evening shares, carrier mix, and early weather. Penalized regressions also receive quadratic terms, numeric interactions and airport interactions. Imputation and scaling are fitted separately inside each training fold. There are **828 candidate configurations** across the two information sets, plus four fixed baselines.

Training fit improves. Future-like validation does not.

Forest with weather • all 22 predictors available at each split



The search did not stop at depth 6.

Tree: 18 depths, up to 30, crossed with four leaf sizes. The best weather tree uses depth 3.

Forest: four predictor counts crossed with seven node sizes, including very shallow trees.

Ridge / LASSO / elastic net: 61 penalties from 0.001 to 1,000,000 and five mixing values.

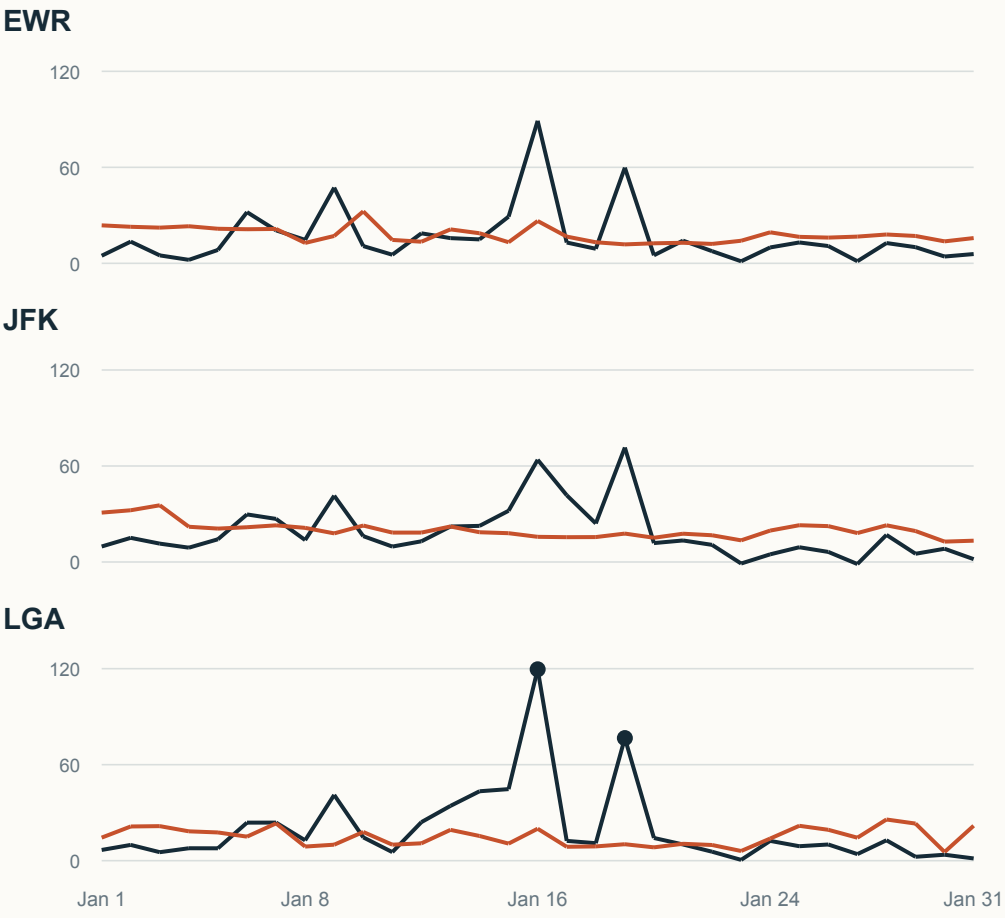
KNN: nine neighbor counts, from 3 to 240. Boundary optima triggered wider grids before evaluation.

The January 2024 outcomes played no role in choosing the method, settings or benchmark. All models were fitted on the full 2023 development sample, and their future predictions were saved before evaluation outcomes were joined.

04 / JANUARY 2024 · 31 DATES · 93 AIRPORT-DAYS

January breaks the winning streak.

Morning weather does not lower future RMSE. Against the airport average, MAE falls from 15.12 to 13.98 minutes, but the largest delays go largely unanticipated. Three dates account for 69.5% of its squared error.



The same difficult dates hit multiple airports. Uncertainty must keep those shared shocks together.

JAN 16 / LGA

120 min

Predicted: 20 min

20.76
AIRPORT-MEAN RMSE

20.89
WEATHER-FOREST RMSE

13.98
WEATHER-FOREST MAE

No demonstrated RMSE gain. The change is **-0.14 minutes** of improvement. A paired seven-day block bootstrap gives a 95% interval of **-1.44 to +1.70 minutes**. The fixed-family forest comparison also straddles zero.

Decision: do not replace the simple benchmark on this evidence. Test more seasons and add forecasts of afternoon weather and network disruption. January has only 31 dates; its 1,025 missing departure outcomes are another blind spot.

Intervals use 2,000 resamples of seven-day blocks, keeping all airports together and the fitted models fixed. RMSE is in minutes of airport-day mean delay, not individual passenger delay. Lower is better.

PROTOCOL.md

Part 1

```
# Protocol fixed before future-period retrieval and afternoon-target modeling

Question: At 09:00 New York time, can airport planners anticipate the mean departure delay among flights scheduled from noon through 23:59 that day? Does pre-09:00 local weather improve predictions beyond calendar and schedule information?

Use 2023 as development; January 2024 is the fixed future evaluation period, selected before retrieval. This is a one-month winter transfer test, not an all-season guarantee. Previous work inspected 2023 daily outcomes, so validation there is development evidence only. January 2024 outcomes will be quarantined until model choices and predictions are locked.

Inputs available by 09:00: airport, day of week, annual sine/cosine, scheduled afternoon flights and route mix, and weather observations from 00:00 through strictly before 08:00 (one-hour reporting buffer). Schedules are reconstructed from final flight records; historical schedule revisions and weather-feed latency cannot be audited. No realized flight-delay input, cancellations, actual departure time, or weather at/after 08:00 enters a predictive model.

Target: mean departure delay among flights scheduled noon-23:59 with observed departure delay. Retain missing outcomes in schedule counts and report coverage; do not impute targets. Exclude no airport-day unless every afternoon outcome is missing. Local dates and scheduled hours define target membership.

Sources: 2023 nycflights23 flight records, 2024 BTS Reporting Carrier monthly file. Download IEM routine hourly airport weather for both periods to use identical time cutoffs, units and observation definitions. Check carrier population against the 2023 snapshot and disclose any source-coverage difference.

Forward validation: train Jan-Mar 2023, validate Apr-May; train Jan-May, validate Jun-Jul; train Jan-Jul, validate Aug-Sep; train Jan-Sep, validate Oct-Dec. No future observations enter a training fold. Select hyperparameters by pooled validation RMSE. All airports on one day remain together. Fold-local imputation/scaling.

Baselines: global mean, airport mean, airport-weekday mean, schedule/calendar OLS. Candidate families: ridge, LASSO, elastic net, KNN, regression tree (depths through 30), random forest. Compare two input sets: calendar/schedule alone and the same inputs plus morning weather. Tune each on the same forward folds. The primary weather increment compares the best separately tuned model for each input set; a paired fixed-family forest ablation is also reported to distinguish information from algorithm choice. No causal interpretation of weather effects.

Lock all selected model settings, baseline and future predictions before joining evaluation outcomes. Refit on all 2023 data with fixed seeds. Evaluate once on January 2024. Report RMSE, MAE, baseline-relative improvement, weather increment, per-airport results, daily error trajectory and largest misses. Use paired 7-day moving-block bootstrap across January dates (2,000 replicates), retaining all airports per sampled date; acknowledge only 31 evaluation dates and few independent weeks. No tuning after evaluation, even if weather adds nothing or hurts.

The final report should answer the operational question, not manufacture a positive result. Retrospective weather associations are development-only and must not be mistaken for causal effects or future predictors.

Source verification before selection: upstream anyflights copies BTS local CRS departure clocks into a UTC-tagged timestamp without converting them. This rebuild uses the original year/month/day and scheduled HHMM fields as local clock values, matching the direct BTS file. The prior daily example's schedule-share features were shifted incorrectly; those results are superseded. Weather for both years is newly retrieved with explicit America/New_York output. Trace precipitation remains 0.0001 inches; it is not treated as missing or exactly zero.

Development-only grid expansion: initial optima reached the largest penalty, largest forest minimum-node size, and largest KNN neighbor count for some families. Before opening future outcomes, expand penalties through 1,000,000; forest minimum-node sizes through 270 (the entire first training sample); and KNN through 240 neighbors (below the first training sample size). For the main operational comparison, allow the best fixed baseline to win the no-weather competition rather than force a weaker tuned model. Retain the fixed-family forest comparison as the weather-information check.
```

download.py

Part 1

```
"""Optional refresh of public source snapshots. Bundled files suffice to reproduce.
Refreshing may change upstream data; use SHA256SUMS to retain the delivered version.
"""
from pathlib import Path
from urllib.request import urlretrieve
from urllib.parse import urlencode
root=Path(__file__).resolve().parent/'data';root.mkdir(exist_ok=True)
urlretrieve('https://raw.githubusercontent.com/moderndive/nycflights23/main/data/flights.rda',root/'flights-2023.rda')
urlretrieve('https://www.transtats.bts.gov/PREZIP/On_Time_Reporting_Carrier_On_Time_Performance_1987_present_2024_1.zip',root/'bts-2024-01.zip')
params=[('station',s) for s in ['EWR','JFK','LGA']]+[('data',v) for v in ['tmpf','relh','sknt','p01i','vsby','mslp']]+list(dict(sts='2023-01-01T05:00:00Z',ets='2024-02-01T05:00:00Z',tz='America/New_York',format='onlycomma',latlon='no',elev='no',missing='M',trace='0.0001',direct='no',report_type='3').items())
urlretrieve('https://mesonet.agron.iastate.edu/cgi-bin/request/asos.py?'+urlencode(params),root/'iem-hourly.csv')
urlretrieve('https://raw.githubusercontent.com/simonpcouch/anyflights/main/R/utils.R',root/'upstream-anyflights-utils.R')
```

prepare.R

Part 1

```
suppressPackageStartupMessages(library(tidyverse))
# BTS CRSDepTime is LOCAL clock time. anyflights copies it into a UTC-tagged
# POSIXct without conversion; do not shift that clock by a timezone offset.
e<-new.env();load('data/flights-2023.rda',e)
f<-e$flights %>% mutate(date=as.Date(sprintf('%04d-%02d-%02d',year,month,day)),
 hour=sched_dep_time%/%100) %>% select(origin,date,hour,dep_delay,distance,carrier)
# Extract only NYC records from the official future-period file. Its outcomes
# are aggregated and quarantined, not printed or used in model selection.
z<-unzip('data/bts-2024-01.zip',list=TRUE);name<-z$Name[which.max(z$Length)]
con<-unz('data/bts-2024-01.zip',name)
g<-read_csv(con,col_types=cols(.default=col_skip(),Origin=col_character(),FlightDate=col_date(),
 CRSDepTime=col_double(),DepDelay=col_double(),Distance=col_double(),Reporting_Airline=col_character
()),progress=FALSE) %>%
 filter(Origin%in%c('EWR','JFK','LGA')) %>%
 transmute(origin=Origin,date=FlightDate,hour=CRSDepTime%/%100,dep_delay=DepDelay,distance=Distance,
carrier=Reporting_Airline)
stopifnot(all(g$date>=as.Date('2024-01-01')&g$date<=as.Date('2024-01-31')),
 all(g$carrier%in%unique(f$carrier)))
f<-bind_rows(f,g)
avg<-function(x)if(all(is.na(x)))NA_real_ else mean(x,na.rm=TRUE)
low<-function(x)if(all(is.na(x)))NA_real_ else min(x,na.rm=TRUE)
high<-function(x)if(all(is.na(x)))NA_real_ else max(x,na.rm=TRUE)
a<-f %>% filter(hour>=12,hour<24) %>% group_by(origin,date) %>% summarize(
 delay=avg(dep_delay),scheduled=n(),distance=avg(distance),longhaul=mean(distance>=1500),
 evening=mean(hour>=17),share_ua=mean(carrier=='UA'),share_b6=mean(carrier=='B6'),
 share_dl=mean(carrier=='DL'),missing_outcomes=sum(is.na(dep_delay)),.groups='drop')
w<-read_csv('data/iem-hourly.csv',na=c('M','','NA'),show_col_types=FALSE,
 col_types=cols(station=col_character(),valid=col_character(),.default=col_double())) %>%
 mutate(date=as.Date(substr(valid,1,10)),hour=as.integer(substr(valid,12,13)))
# Keep routine records exactly once. Clock strings are requested in New York
# local time; 00:00-07:59 never overlaps the noon-or-later target window.
w<-w %>% distinct() %>% mutate(sknt=if_else(sknt<0|sknt>174,NA_real_,sknt),
 p01i=if_else(p01i<0,NA_real_,p01i),vsby=if_else(vsby<0,NA_real_,vsby))
m<-w %>% filter(hour<8) %>% group_by(origin=station,date) %>% summarize(temp=avg(tmpf),
 humidity=avg(relh),wind=avg(sknt),wind_max=high(sknt),rain=avg(p01i),rain_max=high(p01i),
 visibility=avg(vsby),visibility_min=low(vsby),pressure=avg(mslp),weather_hours=n(),.groups='drop')
stopifnot(!anyDuplicated(a[c('origin','date')]),!anyDuplicated(m[c('origin','date')]))
d<-left_join(a,m,by=c('origin','date'),relationship='one-to-one') %>% mutate(
 origin=factor(origin,levels=c('EWR','JFK','LGA')),weekday=factor(as.POSIXlt(date)$wday,levels=0:6),
 sin_year=sin(2*pi*as.numeric(format(date,'%j'))/365.25),cos_year=cos(2*pi*as.numeric(format(date,'%
j'))/365.25),
 weather_missing=as.integer(is.na(weather_hours)))
stopifnot(nrow(d)==nrow(a),!anyNA(d$delay),nrow(d)==396*3)
dev<-filter(d,date<as.Date('2024-01-01'));ev<-filter(d,date>=as.Date('2024-01-01'))
saveRDS(dev,'data/development.rds');saveRDS(select(ev,origin,date,delay,missing_outcomes),'data/eval
uation-labels.rds')
saveRDS(select(ev,-delay,-missing_outcomes),'data/evaluation-features.rds')
write.csv(dev,'results/development.csv',row.names=FALSE)
jsonlite::write_json(list(development_rows=nrow(dev),future_rows=nrow(ev),
 development_flights=sum(dev$scheduled),future_scheduled=sum(ev$scheduled),
 missing_development=sum(dev$missing_outcomes),mean=mean(dev$delay),median=median(dev$delay),
 missing_weather=sum(is.na(d$weather_hours)),weather_records=nrow(w),
 source_carriers_2023=sort(unique(e$flights$carrier)),source_carriers_2024=sort(unique(g$carrier))),
 'results/data-audit.json',auto_unbox=TRUE,pretty=TRUE)
cat('Prepared',nrow(dev),'development rows and',nrow(ev),'future features. No future outcomes
displayed.\n')
```

methods.R

Part 1

```
suppressPackageStartupMessages({library(tidyverse);library(glmnet);library(rpart);library(ranger);library(FNN)})
base_nums<-c('scheduled','distance','longhaul','evening','share_ua','share_b6','share_dl','sin_year','cos_year')
weather_nums<-c('temp','humidity','wind','wind_max','rain','rain_max','visibility','visibility_min','pressure','weather_hours','weather_missing')
features_for<-function(set)c('origin','weekday',base_nums,if(set=='Weather')weather_nums)
prep<-function(a,b,features){for(v in setdiff(features,c('origin','weekday'))){m<-median(a[[v]],na.rm=TRUE);if(!is.finite(m))m<-0;a[[v]][is.na(a[[v]])]<-m;b[[v]][is.na(b[[v]])]<-m};list(a=a,b=b)}
matrices<-function(a,b,features,rich=FALSE){
 x<-model.matrix(reformulate(features),a)[,-1,drop=FALSE];z<-model.matrix(reformulate(features),b)[,-1,drop=FALSE]
 if(rich){ns<-setdiff(features,c('origin','weekday','weather_missing'))
  for(i in seq_along(ns))for(j in i:length(ns)){x<-cbind(x,a[[ns[i]]]*a[[ns[j]]]);z<-cbind(z,b[[ns[i]]]*b[[ns[j]]])}
  for(ap in c('JFK','LGA'))for(v in ns){x<-cbind(x,(a$origin==ap)*a[[v]]);z<-cbind(z,(b$origin==ap)*b[[v]])}
 }
 mu<-colMeans(x);sdv<-apply(x,2,sd);ok<-is.finite(sdv)&sdv>1e-8
 list(x=sweep(sweep(x[,ok,drop=FALSE],2,mu[ok]),2,sdv[ok],'/'),z=sweep(sweep(z[,ok,drop=FALSE],2,mu[ok]),2,sdv[ok],'/'))
}
group_pred<-function(a,b,keys){key<-function(x)do.call(paste,c(x[keys],sep='|'));t<-tapply(a$delay,key(a),mean);p<-as.numeric(t[key(b)]);p[is.na(p)]<-mean(a$delay);p}
ols<-function(a,b){f<-reformulate(features_for('Schedule'));x<-model.matrix(f,a);z<-model.matrix(f,b);q<-qr(x);ok<-q$pivot[seq_len(q$rank)];as.numeric(z[,ok,drop=FALSE]%*%lm.fit(x[,ok,drop=FALSE],a$delay)$coefficients)}
baselines<-function(a,b)list('Global mean'=rep(mean(a$delay),nrow(b)),'Airport mean'=group_pred(a,b,'origin'),
 'Airport-weekday'=group_pred(a,b,c('origin','weekday')),'Schedule OLS'=ols(a,b))
fit_predict<-function(cfg,a,b,set,seed=242){
 features<-features_for(set);p<-prep(a,b,features);a<-p$a;b<-p$b;f<-reformulate(features,response='delay')
 family<-cfg$family
 if(family%in%c('Ridge','LASSO','Elastic net')){m<-matrices(a,b,features,TRUE);fit<-glmnet(m$x,a$delay,alpha=cfg$alpha,lambda=cfg$lambda,standardize=FALSE);return(as.numeric(predict(fit,m$z,s=cfg$lambda)))}
 if(family=='KNN'){m<-matrices(a,b,features);return(FNN::knn.reg(m$x,m$z,a$delay,k=cfg$k)$pred)}
 if(family=='Tree'){fit<-rpart(f,data=a,method='anova',control=rpart.control(cp=0,maxdepth=cfg$depth,minbucket=cfg$leaf,minsplit=2*cfg$leaf,xval=0));return(as.numeric(predict(fit,b)))}
 fit<-ranger(f,data=a,num.trees=500,mtry=cfg$mtry,min.node.size=cfg$leaf,respect.unordered.factors='order',seed=seed,num.threads=2)
 predict(fit,b)$predictions
}
```

select.R

Part 1

```
source('methods.R')
d<-readRDS('data/development.rds')
ends<-as.Date(c('2023-03-31','2023-05-31','2023-07-31','2023-09-30'))
stops<-as.Date(c('2023-05-31','2023-07-31','2023-09-30','2023-12-31'))
registry<-list();results<-list();predictions<-list();ii<-0;pp<-0
record<-function(cfg,set,fold,a,b,p,pt){
 ii<<-ii+1;results[[ii]]<<-
tibble(id=cfg$id,set=set,family=cfg$family,fold=fold,n=nrow(b),sse=sum((b$delay-
p)^2),sae=sum(abs(b$delay-p)),train_n=nrow(a),train_sse=sum((a$delay-pt)^2))
 pp<<-pp+1;predictions[[pp]]<<-
tibble(id=cfg$id,set=set,fold=fold,origin=b$origin,date=b$date,observed=b$delay,predicted=p)
}
for(set in c('Schedule','Weather')){
 features<-features_for(set)
 reg<-bind_rows(expand_grid(alpha=c(0,.25,.5,.75,1),lambda=10^seq(6,-
3,length.out=61))%>%mutate(family=case_when(alpha==0~'Ridge',alpha==1~'LASSO',TRUE~'Elastic net')),
 expand_grid(depth=c(1:15,20,25,30),leaf=c(5,10,20,40))%>%mutate(family='Tree'),
 expand_grid(mtry=unique(c(1,3,6,length(features))),leaf=c(1,5,15,30,60,120,270))%>%mutate(family='F
orest'),
 tibble(k=c(3,5,10,20,40,80,120,160,240),family='KNN'))%>%mutate(id=row_number(),set=set)
 registry[[set]]<-reg
 for(fold in 1:4){
  a<-filter(d,date<=ends[fold]);b<-
filter(d,date>ends[fold],date<=stops[fold]);stopifnot(max(a$date)<min(b$date))
  p<-prep(a,b,features);a<-p$a;b<-p$b
  if(set=='Schedule'){bp<-baselines(a,b);bt<-baselines(a,a)
   for(n in
names(bp))record(list(id=-match(n,names(bp)),family=n),'Baseline',fold,a,b,bp[[n]],bt[[n]])}
  m<-matrices(a,b,features,TRUE)
  for(al in c(0,.25,.5,.75,1)){
   cfg<-filter(reg,!is.na(alpha),alpha==al)%>%arrange(desc(lambda))
   fit<-glmnet(m$x,a$delay,alpha=al,lambda=cfg$lambda,standardize=FALSE)
   pv<-predict(fit,m$z,s=cfg$lambda);pt<-predict(fit,m$x,s=cfg$lambda)
   for(j in 1:nrow(cfg))record(cfg[j,],set,fold,a,b,pv[,j],pt[,j])
  }
  m<-matrices(a,b,features);ks<-filter(reg,family=='KNN')
  nn<-FNN::get.knnx(m$x,m$z,k=max(ks$k))$nn.index;nt<-FNN::get.knnx(m$x,m$x,k=max(ks$k))$nn.index
  for(j in 1:nrow(ks)){k<-ks$k[j];pv<-
rowMeans(matrix(a$delay[nn[,1:k,drop=FALSE]],nrow=nrow(b)));pt<-
rowMeans(matrix(a$delay[nt[,1:k,drop=FALSE]],nrow=nrow(a)));record(ks[j,],set,fold,a,b,pv,pt)}
  for(j in
which(reg$family%in%c('Tree','Forest'))){cfg<-reg[j,];f<-reformulate(features,response='delay')
   if(cfg$family=='Tree'){fit<-
rpart(f,data=a,method='anova',control=rpart.control(cp=0,maxdepth=cfg$depth,minbucket=cfg$leaf,minsp
lit=2*cfg$leaf,xval=0));pv<-predict(fit,b);pt<-predict(fit,a)}else{
    fit<-
ranger(f,data=a,num.trees=500,mtry=cfg$mtry,min.node.size=cfg$leaf,respect.unordered.factors='order'
,seed=242+fold,num.threads=2);pv<-predict(fit,b)$predictions;pt<-predict(fit,a)$predictions}
   record(cfg,set,fold,a,b,pv,pt)
  }
  cat(set,'forward fold',fold,'complete\n');flush.console()
 }
}
reg<-bind_rows(registry);raw<-bind_rows(results);pr<-bind_rows(predictions)
score<-
raw%>%group_by(set,id,family)%>%summarize(RMSE=sqrt(sum(sse)/sum(n)),MAE=sum(sae)/sum(n),train_RMSE=
sqrt(sum(train_sse)/sum(train_n)),.groups='drop')%>%left_join(select(reg,-
family),by=c('set','id'))%>%arrange(RMSE)
board<-score%>%group_by(set,family)%>%slice_min(RMSE,n=1,with_ties=FALSE)%>%ungroup()
chosen<-board%>%group_by(set)%>%slice_min(RMSE,n=1,with_ties=FALSE)%>%ungroup()
write.csv(reg,'results/configurations.csv',row.names=FALSE);write.csv(raw,'results/fold-
scores.csv',row.names=FALSE)
write.csv(score,'results/cv-
grid.csv',row.names=FALSE);write.csv(board,'results/leaderboard.csv',row.names=FALSE)
write.csv(chosen,'results/selected.csv',row.names=FALSE);write.csv(pr,'results/validation-
predictions.csv',row.names=FALSE)
write.csv(tibble(train_end=ends,validation_end=stops),'results/forward-folds.csv',row.names=FALSE)
print(chosen);print(board)
```

lock.R

Part 1

```
source('methods.R')
d<-readRDS('data/development.rds');x<-readRDS('data/evaluation-features.rds')
stopifnot(!any(c('delay','missing_outcomes')%in%names(x)),max(d$date)<min(x$date))
b<-read.csv('results/leaderboard.csv')
no<-filter(b,set%in%c('Schedule','Baseline'))%>%arrange(RMSE,id)%>%slice_head(n=1)
yes<-filter(b,set=='Weather')%>%arrange(RMSE,id)%>%slice_head(n=1)
fs<-filter(b,set=='Schedule',family=='Forest');fw<-filter(b,set=='Weather',family=='Forest')
choices<-bind_rows(mutate(no,role='No weather'),mutate(yes,role='Morning
weather'),mutate(fs,role='Forest without weather'),mutate(fw,role='Forest with weather'))
write.csv(choices,'results/selection-lock.csv',row.names=FALSE)
p<-select(x,origin,date)
for(i in 1:nrow(choices)){
 cfg<-choices[i,]
 if(cfg$set=='Baseline')
{ab<-prep(d,x,features_for('Schedule'));pred<-baselines(ab$a,ab$b)[[cfg$family]]}
 else pred<-fit_predict(cfg,d,x,cfg$set,seed=2422024)
 p[[cfg$role]]<-pred
}
write.csv(p,'results/locked-predictions.csv',row.names=FALSE)
writeLines(c(paste('protocol',tools::md5sum('PROTOCOL.md')),paste('selection',tools::md5sum('results
/selection-lock.csv')),
 paste('predictions',tools::md5sum('results/locked-predictions.csv'))),'results/before-evaluation-
hashes.txt')
cat('Locked choices and predictions without reading future outcome file.\n');print(choices)
```

evaluate.R

Part 1

```
suppressPackageStartupMessages(library(tidyverse))
stopifnot(file.exists('results/before-evaluation-hashes.txt'))
p<-read.csv('results/locked-predictions.csv',check.names=FALSE)%>%mutate(date=as.Date(date))
e<-left_join(p,readRDS('data/evaluation-labels.rds'),by=c('origin','date'),relationship='one-to-one')
stopifnot(nrow(e)==93,!anyNA(e$delay))
roles<-c('No weather','Morning weather','Forest without weather','Forest with weather')
score<-function(d,col)sqrt(mean((d$delay-d[[col]])^2))
s<-map_dfr(roles,function(v)tibble(role=v,RMSE=score(e,v),MAE=mean(abs(e$delay-e[[v]]))))
write.csv(e,'results/evaluation.csv',row.names=FALSE);write.csv(s,'results/evaluation-scores.csv',row.names=FALSE)
a<-e%>%group_by(origin)%>%summarize(no_weather=sqrt(mean((delay-`No weather`)^2)),weather=sqrt(mean((delay-`Morning weather`)^2)),MAE=mean(abs(delay-`Morning weather`)),.groups='drop')
write.csv(a,'results/airport-errors.csv',row.names=FALSE)
set.seed(2422025);dates<-sort(unique(e$date));n<-length(dates)
boot<-map_dfr(1:2000,function(i){starts<-sample(1:(n-6),ceiling(n/7),replace=TRUE);ix<-unlist(lapply(starts,function(s)s:(s+6)))[1:n];ix<-unlist(lapply(dates[ix],function(t)which(e$date==t)));z<-e[ix,];b<-score(z,'No weather');m<-score(z,'Morning weather');f<-score(z,'Forest without weather');g<-score(z,'Forest with weather');tibble(gain=b-m,percent=100*(1-m/b),forest_gain=f-g,model_rmse=m)})
write.csv(boot,'results/block-bootstrap.csv',row.names=FALSE)
jsonlite::write_json(list(model_rmse=s$RMSE[2],no_weather_rmse=s$RMSE[1],mae=s$MAE[2],
 gain=s$RMSE[1]-s$RMSE[2],gain_percent=100*(1-s$RMSE[2]/s$RMSE[1]),

gain_interval=quantile(boot$gain,c(.025,.975)),percent_interval=quantile(boot$percent,c(.025,.975)),
 forest_gain=s$RMSE[3]-s$RMSE[4],forest_gain_interval=quantile(boot$forest_gain,c(.025,.975)),
 missing_outcomes=sum(e$missing_outcomes),mean_delay=mean(e$delay)),
 'results/evaluation-summary.json',pretty=TRUE,auto_unbox=TRUE,digits=8)
print(s);print(a);print(quantile(boot$percent,c(.025,.975)))
```

describe.R

Part 1

```
suppressPackageStartupMessages(library(tidyverse))
d<-readRDS('data/development.rds') %>% mutate(month=as.integer(format(date,'%m')),
 quarter=paste0('Q',ceiling(month/3)),rough=visibility_min<3|wind_max>=20,
 condition=if_else(rough,'Low visibility / high wind','Other mornings'))
write.csv(d,'results/development-visuals.csv',row.names=FALSE)
# Predefined weather cutoffs: minimum visibility <3 miles or max wind >=20 knots.
# Descriptive contrasts are not causal effects; show all airport-quarter cells.
contrasts<-d %>% filter(!is.na(rough)) %>% group_by(origin,quarter,rough) %>%
 summarize(n=n(),mean=mean(delay),median=median(delay),lo=quantile(delay,.25),hi=quantile(delay,.75)
,.groups='drop')
write.csv(contrasts,'results/weather-contrasts.csv',row.names=FALSE)
monthly<-d %>% group_by(origin,month) %>% summarize(mean=mean(delay),median=median(delay),lo=quantil
e(delay,.25),hi=quantile(delay,.75),.groups='drop')
write.csv(monthly,'results/monthly.csv',row.names=FALSE)
```

finish.R

Part 1

```
suppressPackageStartupMessages(library(tidyverse))
d<-readRDS('data/development.rds');x<-readRDS('data/evaluation-features.rds');e<-read.csv('results/evaluation.csv',check.names=FALSE)%>%mutate(date=as.Date(date))
lock<-read.csv('results/selection-lock.csv');v<-read.csv('results/validation-predictions.csv');fold<-read.csv('results/fold-scores.csv');reg<-read.csv('results/configurations.csv')
# Extract paired predictions using choices fixed before evaluation.
p<-map_dfr(1:nrow(lock),function(i){z<-lock[i,];v%>%filter(set==z$set,id==z$id)%>%mutate(role=z$role)})
write.csv(p,'results/chosen-validation-predictions.csv',row.names=FALSE)
f<-p%>%group_by(role,fold)%>%summarize(n=n(),RMSE=sqrt(mean((observed-predicted)^2)),.groups='drop')
write.csv(f,'results/forward-comparison.csv',row.names=FALSE)
daily<-e%>%group_by(date)%>%summarize(mean_observed=mean(delay),mean_predicted=mean(`Morning weather`),
 sse=sum((delay-`Morning weather`)^2),baseline_sse=sum((delay-`No weather`)^2),.groups='drop')
write.csv(daily,'results/future-daily.csv',row.names=FALSE)
worst<-e%>%mutate(error=`Morning weather`-delay)%>%arrange(desc(abs(error)))
write.csv(head(worst,6),'results/worst-errors.csv',row.names=FALSE)
jsonlite::write_json(list(configurations=nrow(reg),fold_fits=nrow(fold),
 validation_no=lock$RMSE[1],validation_weather=lock$RMSE[2],
 validation_gain=100*(1-lock$RMSE[2]/lock$RMSE[1]),
 top3_dates_sse_share=sum(sort(daily$sse,decreasing=TRUE)[1:3])/sum(daily$sse),
 morning_weather_days=sum(with(d,visibility_min<3|wind_max>=20),na.rm=TRUE)),
 'results/findings.json',auto_unbox=TRUE,pretty=TRUE,digits=8)
# Check splits, paired baseline construction, complete grids and frozen hashes.
stopifnot(nrow(d)==1095,nrow(x)==93,max(d$date)<min(x$date),
 all(table(x$date)==3),!any(c('delay','missing_outcomes')%in%names(x)),
 all(table(interaction(fold$set,fold$id,drop=TRUE))==4),
 !anyDuplicated(reg[c('set','id')]))
means<-tapply(d$delay,d$origin,mean)
stopifnot(max(abs(e$`No weather`-means[e$origin]))<1e-10)
h<-readLines('results/before-evaluation-hashes.txt')
stopifnot(h[1]==paste('protocol',tools::md5sum('PROTOCOL.md')),
 h[2]==paste('selection',tools::md5sum('results/selection-lock.csv')),
 h[3]==paste('predictions',tools::md5sum('results/locked-predictions.csv')))
for(i in 1:nrow(lock)){
 z<-lock[i,];o<-filter(p,role==z$role)
 stopifnot(abs(sqrt(mean((o$observed-o$predicted)^2))-z$RMSE)<1e-9)
}
writeLines(c('PASS: all training dates precede every future date.',
 'PASS: complete four-fold grid; no duplicated candidate keys.',
 'PASS: no future outcome or missing-outcome-count feature.',
 'PASS: future baseline equals development-only airport means.',
 'PASS: selection and prediction locks unchanged after evaluation.',
 'PASS: pooled validation metrics recomputed from paired predictions.'),'results/audit.txt')
print(readLines('results/audit.txt'))
```

example.Rmd

Part 1

```
---
title: "The 9 a.m. forecast"
author: "EC242 | Instructor-created example"
output: md_document
params:
  show_code: false
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo=params$show_code, message=FALSE, warning=FALSE)
```

At 09:00, can morning weather improve predictions of afternoon departure delay?
The report uses 2023 for forward validation and January 2024 for a locked future
evaluation. The rendered report reads audited results; document changes cannot
silently change the selected model or evaluation sample.

```{r vector-report, results='hide'}
m <- jsonlite::read_json('results/evaluation-summary.json')
stopifnot(is.finite(m$model_rmse))
python <- Sys.getenv('PYTHON', unset=Sys.which('python3'))
output <- if(params$show_code) 'example-with-code.pdf' else 'example-memo.pdf'
stopifnot(system2(python,c('report.py','--output',output,
                           if(params$show_code)'--code'))==0)
```

The vector renderer supplies identical report pages in both editions. When code
is on, it appends the complete analysis and rendering scripts with line numbers.
All raw data and computed results are supplied in the source archive.
```

render.R

Part 1

```
1  for(show in c(FALSE,TRUE))rmarkdown::render('example.Rmd',params=list(show_code=show),quiet=TRUE,
2   output_file=if(show)'example-with-code.md' else 'example-
   memo.md',envir=new.env(parent=globalenv()))
3  writeLines(capture.output(sessionInfo()),'session-info.txt')
```

reproduce.R

Part 1

```
# Reproduce the registered experiment; this is not a new unopened evaluation.
for(script in c('prepare.R','select.R','lock.R','evaluate.R','describe.R','finish.R','render.R')){
 cat('Running',script,'\n')
 stopifnot(system2(file.path(R.home('bin'),'Rscript'),c('--vanilla',script))==0)
}
```

report.py

Part 1

```
"""Publication-style vector report. Charts use only saved, audited analysis exports."""
from pathlib import Path
import csv,json,math,calendar,datetime,textwrap,argparse
from reportlab.pdfgen import canvas
from reportlab.pdfbase import pdfmetrics
from reportlab.pdfbase.ttfonts import TTFont
from reportlab.lib.colors import HexColor
from reportlab.lib.styles import ParagraphStyle
from reportlab.platypus import Paragraph
ROOT=Path(__file__).resolve().parent; D=ROOT/'results'
def rows(n):return list(csv.DictReader((D/n).open()))
def val(z,k):return float(z[k])
def read(n):return json.loads((D/n).read_text())
A=read('data-audit.json');M=read('evaluation-summary.json');F=read('findings.json')
DEV=rows('development-visuals.csv');CON=rows('weather-contrasts.csv');EV=rows('evaluation.csv')
BOARD=rows('leaderboard.csv');GRID=rows('cv-grid.csv');FORWARD=rows('forward-comparison.csv');AIR=rows('airport-errors.csv');WORST=rows('worst-errors.csv')
INK='#152A36';MUTED='#667680';PAPER='#FCFBF7';BLUE='#176EAD';TEAL='#007F76';ORANGE='#C6512C';LINE='#D9DEDD';PALE='#E8EFEE';WHITE='#FFFFFF'
for name,file,fallback in [('Sans','Arial.ttf','Helvetica'),('Bold','Arial Bold.ttf','Helvetica-Bold'),('Serif','Georgia.ttf','Times-Roman'),('Mono','Courier New.ttf','Courier')]:
 p=Path('/System/Library/Fonts/Supplemental')/file
 if p.exists():pdfmetrics.registerFont(TTFont(name,str(p)))
 else:pdfmetrics.registerFont(pdfmetrics.Font(name,fallback,'WinAnsiEncoding'))
pdfmetrics.registerFontFamily('Sans',normal='Sans',bold='Bold',italic='Sans',boldItalic='Bold')
class Report:
 def __init__(self,path):
  self.c=canvas.Canvas(str(path),pagesize=(612,792),pageCompression=1);self.page=0
  self.c.setTitle('The 9 a.m. forecast | Morning weather meets a future test')
  self.c.setAuthor('EC242 | Instructor-created example');self.c.setSubject('Forward validation, weather ablation, and a locked January 2024 evaluation')
 def rect(self,x,y,w,h,c):self.c.setFillColor(HexColor(c));self.c.rect(x,792-y-h,w,h,fill=1,stroke=0)
 def line(self,x,y,u,v,c=LINE,width=.6,dash=None):
  self.c.setStrokeColor(HexColor(c));self.c.setLineWidth(width);self.c.setDash(dash or []);self.c.line(x,792-y,u,792-v);self.c.setDash([])
 def text(self,x,y,t,size=10,font='Sans',c=INK,align='left'):
  self.c.setFillColor(HexColor(c));self.c.setFont(font,size)
  {'left':self.c.drawString,'right':self.c.drawRightString,'center':self.c.drawCentredString}[align](x,792-y-size*.82,str(t))
 def para(self,x,y,w,t,size=10,leading=14,c=INK,limit=200):
  p=Paragraph(t,ParagraphStyle('p',fontName='Sans',fontSize=size,leading=leading,textColor=HexColor(c)))
  _,h=p.wrap(w,740);assert h<=limit,(self.page,y,h,limit,t[:60]);assert y+h<746,(self.page,y,h)
  p.drawOn(self.c,x,792-y-h);return y+h
 def dot(self,x,y,r=3,c=TEAL,hollow=False):
  self.c.setFillColor(HexColor(PAPER if hollow else c));self.c.setStrokeColor(HexColor(c));self.c.setLineWidth(1.1);self.c.circle(x,792-y,r,stroke=int(hollow),fill=1)
 def start(self,section):
  if self.page:self.c.showPage()
  self.page+=1;self.rect(0,0,612,792,PAPER);self.rect(36,24,17,4,ORANGE)
  self.text(62,20,'EC242 / INSTRUCTOR-CREATED EXAMPLE',7.5,'Bold');self.text(576,20,section.upper(),7.5,'Sans',MUTED,'right');self.line(36,42,576,42)
  self.c.bookmarkPage(str(self.page));self.c.addOutlineEntry(section,str(self.page),0)
  self.line(36,751,576,751);self.text(36,764,'THE 9 A.M. FORECAST',7,'Bold',MUTED);self.text(528,764,'NYC / 2023-24',7,'Sans',MUTED,'right');self.text(576,759,f'{self.page:02}',13,'Bold',ORANGE,'right')
 def title(self,k,t,sub):
  self.text(36,62,k,8,'Bold',ORANGE);self.text(36,84,t,min(29,29*540/pdfmetrics.stringWidth(t,'Serif',29)),'Serif');self.para(36,127,540,sub,10,14,limit=58)
 def note(self,y,t):self.para(36,y,540,t,8,11,MUTED,limit=55)
 def save(self):self.c.save()

def opening(r):
 r.start('Question + data')
 r.text(36,64,'AN AIRPORT-DAY FORECAST, BUILT BEFORE LUNCH',8,'Bold',ORANGE)
 r.text(36,90,'The 9 a.m. forecast',40,'Serif')
 r.para(36,153,540,'Can morning weather help airport planners anticipate afternoon departure delays?
```

report.py

Part 2

```
     We combine flight schedules with weather recorded before 8 a.m., predict the mean delay of flights
    scheduled from noon onward, and test the model on a later month. Weather cuts validation error by
    <b>12.3%</b> in 2023. In January 2024, that advantage disappears. A few severe days overwhelm the
    forecast.',11,15.5,limit=80)
56   # A genuine information boundary: observed inputs, forecast issued, target window.
57   r.rect(36,252,540,139,INK)
58   r.text(52,268,'THE CLOCK IS PART OF THE MODEL',8,'Bold','#B6C9CE')
59   x=lambda h:57+h/24*499
60   r.rect(x(0),306,x(8)-x(0),16,TEAL);r.rect(x(8),306,x(9)-x(8),16,'#527078');r.rect(x(12),306,x(24)-
    x(12),16,ORANGE)
61   r.line(x(9),298,x(9),345,'#FFFFFF',1.2)
62   for h,t in
    [(0,'00:00'),(8,'08:00'),(12,'12:00'),(24,'24:00')]:r.text(x(h),288,t,8,'Bold',WHITE,'right' if
    h==24 else 'left')
63   r.text(57,336,'Observed weather',9,'Bold',WHITE);r.text(x(12),336,'Flights we
    predict',9,'Bold',WHITE)
64   r.text(52,365,'09:00 / ISSUE FORECAST',8,'Bold','#D4E2E3');r.text(560,365,'A one-hour weather
    reporting buffer',8,'Sans','#D4E2E3','right')
65   r.text(36,418,'Data',17,'Serif')
66   r.para(36,447,259,'We combine <b>BTS flight records</b> with <b>Iowa Environmental Mesonet weather
    observations</b>. The 2023 flights come through nycflights23; January 2024 comes directly from BTS.
    Both use the same reporting-carrier definitions and contain the same 14 carriers. We aggregate each
    source by airport and local date, then check a one-to-one join. The panel contains <b>1,188 airport-
    days</b>: 1,095 for development and 93 for the future test.',9.4,12.6,limit=145)
67   r.para(317,447,259,'The outcome includes early departures and averages only recorded delays. Of
    262,528 scheduled afternoon flights in development, <b>8,180 lack a delay outcome</b>; they remain
    in schedule counts. Development mean delay is <b>21.1 minutes</b>, versus a median of <b>12.8</b>.
    Weather is available for every airport-day; missing individual readings use training-only medians.
    Trace rain remains a small positive amount. No afternoon weather enters the
    forecast.',9.4,12.6,limit=145)
68   r.line(36,600,576,600)
69   for x,n,l in [(36,'1,095','DEVELOPMENT AIRPORT-DAYS'),(222,'93','FUTURE AIRPORT-
    DAYS'),(408,'08:00','WEATHER CUTOFF')]:
70     r.text(x,617,n,30,'Bold',TEAL);r.text(x,657,l,7.2,'Bold',MUTED)
71   r.note(689,'Schedules are reconstructed from final records, so historical schedule revisions and
    feed latency remain untested. Missing delay outcomes, including cancellations, are a separate
    operational risk. January supplies 7.8% of the combined panel.')
72   r.text(36,732,'Sources: BTS / nycflights23 / Iowa Environmental Mesonet',7,'Sans',BLUE)
73   r.c.linkURL('https://www.transtats.bts.gov/PREZIP/',(36,51,160,63),relative=0)
74   r.c.linkURL('https://moderndive.github.io/nycflights23/',(160,51,270,63),relative=0)
75   r.c.linkURL('https://mesonet.agron.iastate.edu/request/download.phtml',(270,51,576,63),relative=0)
76
77  def shade(value):
78   # Monotone light-to-dark palette; the legend explicitly caps high values.
79   t=max(0,min(1,value/90));stops=[(0,(239,242,236)),(.3,(151,191,191)),(.65,(41,120,140)),(1,(19,47,7
    4))]
80   for (a,c),(b,d) in zip(stops,stops[1:]):
81    if a<=t<=b:
82     q=(t-a)/(b-a);return '#'+''.join(f'{round(u+(v-u)*q):02x}' for u,v in zip(c,d))
83   return '#132F4A'
84
85  def patterns(r):
86   r.start('Patterns in development')
87   r.title('01 / SEASON AND WEATHER','Bad mornings are not the whole story.','Delay clusters in
    summer, including days without low visibility or strong morning wind. Weather matters differently
    across airports and seasons.')
88   r.text(36,187,'Every afternoon in 2023',13,'Bold');r.text(576,191,'Mean departure delay,
    minutes',8,'Sans',MUTED,'right')
89   for j,ap in enumerate(['EWR','JFK','LGA']):
90    x0=57+j*182;y0=242;cw=12.2;ch=5.2
91    r.text(x0,216,ap,12,'Bold')
92    for m in range(1,13):r.text(x0+(m-.5)*cw,233,calendar.month_abbr[m][0],6.8,'Sans',MUTED,'center')
93    for day in [1,10,20,31]:r.text(x0-7,y0+(day-1)*ch,str(day),6.5,'Sans',MUTED,'right')
94    for z in [a for a in DEV if a['origin']==ap]:
95     dt=datetime.date.fromisoformat(z['date']);r.rect(x0+(dt.month-1)*cw,y0+(dt.day-1)*ch,cw-.9,ch-
    .7,shade(val(z,'delay')))
96   for i,t in enumerate([0,15,30,60,90]):r.rect(171+i*48,421,19,8,shade(t));r.text(194+i*48,420,'90+'
    if t==90 else ('≤0' if t==0 else str(t)),7,'Sans',MUTED)
```

report.py

Part 3

```
 r.text(36,420,'KEY',7,'Bold',MUTED)
 r.text(36,460,'Within the same airport and quarter',13,'Bold')
 r.text(36,482,'Mean afternoon delay: other mornings ○  →  low visibility / high wind ●',8.5,'Sans',MUTED)
 # Twelve comparisons share the same 0-60-minute scale; never hide reversals.
 for j,ap in enumerate(['EWR','JFK','LGA']):
  yy=524+j*53;r.text(36,yy+3,ap,11,'Bold')
  for k,q in enumerate(['Q1','Q2','Q3','Q4']):
   x0=109+k*117;xx=lambda v:x0+v/60*90
   a=next(z for z in CON if z['origin']==ap and z['quarter']==q and z['rough']=='FALSE')
   b=next(z for z in CON if z['origin']==ap and z['quarter']==q and z['rough']=='TRUE')
   if j==0:r.text(x0+45,506,q,8,'Bold',MUTED,'center')
   r.line(xx(0),yy+10,xx(60),yy+10,LINE,1)
   r.line(xx(val(a,'mean')),yy+10,xx(val(b,'mean')),yy+10,ORANGE,1.8)
   r.dot(xx(val(a,'mean')),yy+10,3.2,TEAL,True);r.dot(xx(val(b,'mean')),yy+10,3.2,ORANGE)
   r.text(x0,yy+24,f"{val(a,'mean'):.0f} → {val(b,'mean'):.0f} min",7.5,'Bold')
   r.text(x0+96,yy+24,f"n={b['n']}",7,'Sans',MUTED,'right')
 r.note(689,'Weather flag: minimum pre-8 a.m. visibility below 3 miles or maximum wind at least 20 knots. Counts show flagged mornings; some cells are small. Mean delay is higher in 10 of 12 comparisons, not all 12. These are descriptive associations, not causal estimates.')

def validation(r):
 r.start('What weather adds')
 r.title('02 / PAIRED FORWARD COMPARISONS','Give weather a fair test.','Tune each method twice: once with calendar and schedule inputs, then with those inputs plus morning weather. Both versions predict the same later dates.')
 r.text(36,184,'The validation clock only moves forward',13,'Bold')
 x0,x1=170,576;xx=lambda m:x0+(m-1)/12*(x1-x0)
 for i,(end,last) in enumerate([(3,5),(5,7),(7,9),(9,12)]):
  y=216+i*24;r.text(36,y+3,f'ROUND {i+1}',7.5,'Bold',MUTED);r.rect(xx(1),y,xx(end+1)-xx(1)-2,13,INK);r.rect(xx(end+1),y,xx(last+1)-xx(end+1)-2,13,ORANGE)
 for m in range(1,13):r.text(xx(m)+11,318,calendar.month_abbr[m][0],7,'Sans',MUTED,'center')
 r.rect(36,318,10,6,INK);r.text(51,316,'Train',7,'Sans',MUTED);r.rect(94,318,10,6,ORANGE);r.text(109,316,'Validate',7,'Sans',MUTED)
 r.text(36,357,'Weather helps more than changing the algorithm.',13,'Bold')
 r.text(36,381,'Pooled forward-validation RMSE, minutes • lower is better',8.5,'Sans',MUTED)
 xx=lambda v:230+(v-24)/9*300
 for t in [24,26,28,30,32]:r.line(xx(t),415,xx(t),586);r.text(xx(t),599,str(t),8,'Sans',MUTED,'center')
 for i,fam in enumerate(['Forest','Tree','LASSO','Elastic net','Ridge','KNN']):
  y=420+i*31;a=next(z for z in BOARD if z['set']=='Schedule' and z['family']==fam);b=next(z for z in BOARD if z['set']=='Weather' and z['family']==fam)
  r.text(36,y-5,'Random forest' if fam=='Forest' else fam,10,'Bold' if i==0 else 'Sans')
  r.line(xx(val(a,'RMSE')),y,xx(val(b,'RMSE')),y,TEAL,2)
  r.dot(xx(val(a,'RMSE')),y,4,MUTED,True);r.dot(xx(val(b,'RMSE')),y,4,TEAL)
  r.text(xx(val(b,'RMSE'))-9,y-4,f"{val(b,'RMSE'):.2f}",8,'Bold',TEAL,'right')
  r.text(xx(val(a,'RMSE'))+9,y-4,f"{val(a,'RMSE'):.2f}",8,'Sans',MUTED)
 r.text(230,622,'● Morning weather',8,'Bold',TEAL);r.text(386,622,'○ Calendar + schedule only',8,'Sans',MUTED)
 r.para(36,648,540,'The strongest no-weather choice is the airport average: <b>29.84 minutes RMSE</b>. The weather forest lowers that to <b>26.18</b>, or <b>12.3%</b>. But the gain varies by season:',9.5,13,limit=40)
 for i,(period,fold) in enumerate([('APR-MAY',1),('JUN-JUL',2),('AUG-SEP',3),('OCT-DEC',4)]):
  x=36+i*139;a=next(z for z in FORWARD if z['role']=='No weather' and int(z['fold'])==fold);b=next(z for z in FORWARD if z['role']=='Morning weather' and int(z['fold'])==fold)
  c=TEAL if val(b,'RMSE')<val(a,'RMSE') else ORANGE
  r.text(x,692,period,7.5,'Bold',MUTED);r.text(x,711,f"{val(a,'RMSE'):.1f} → {val(b,'RMSE'):.1f}",16,'Bold',c)
 r.text(36,735,'Airport-average → morning-weather RMSE, minutes. Each window is later than its training data.',7,'Sans',MUTED)

def tuning(r):
 r.start('Model specification + tuning')
 r.title('03 / SELECT BEFORE TESTING','A restrained forest wins.', 'The selected model averages 500 trees, considers all 22 predictors at a split, and stops splitting nodes smaller than 30 observations. All tuning uses 2023 only.')
 r.para(36,181,540,'Inputs cover airport, weekday, annual seasonality, afternoon schedule size, route distance, long-haul and evening shares, carrier mix, and early weather. Penalized regressions
```

report.py

Part 4

```
    also receive quadratic terms, numeric interactions and airport interactions. Imputation and scaling
    are fitted separately inside each training fold. There are <b>828 candidate configurations</b>
    across the two information sets, plus four fixed baselines.',9.5,13,limit=65)
147   r.text(36,266,'Training fit improves. Future-like validation does not.',13,'Bold')
148   r.text(36,289,'Forest with weather • all 22 predictors available at each split',8.5,'Sans',MUTED)
149   vals=sorted([z for z in GRID if z['set']=='Weather' and z['family']=='Forest' and
    val(z,'mtry')==22],key=lambda z:-val(z,'leaf'))
150   x0,x1=73,561;y0,y1=325,478;xx=lambda i:x0+i*(x1-x0)/(len(vals)-1);yy=lambda v:y1-v/40*(y1-y0)
151   for t in [0,10,20,30,40]:r.line(x0,yy(t),x1,yy(t));r.text(61,yy(t)-4,str(t),8,'Sans',MUTED,'right')
152   for key,color in [('RMSE',ORANGE),('train_RMSE',TEAL)]:
153     for i in
    range(len(vals)-1):r.line(xx(i),yy(val(vals[i],key)),xx(i+1),yy(val(vals[i+1],key)),color,2)
154     for i,z in enumerate(vals):r.dot(xx(i),yy(val(z,key)),3,color)
155   for i,z in enumerate(vals):r.text(xx(i),488,z['leaf'],8,'Sans',MUTED,'center')
156   best=min(range(len(vals)),key=lambda
    i:val(vals[i],'RMSE'));r.dot(xx(best),yy(val(vals[best],'RMSE')),6,ORANGE,True)
157   r.text(xx(best),yy(val(vals[best],'RMSE'))-21,'SELECTED / 26.18',8,'Bold',ORANGE,'center')
158   r.text(73,511,'Minimum node size: large → small / increasing flexibility',8,'Bold')
159   r.text(370,315,'Validation',8,'Bold',ORANGE);r.text(478,315,'Training',8,'Bold',TEAL)
160   r.text(36,552,'The search did not stop at depth 6.',13,'Bold')
161   r.para(36,579,259,'<b>Tree:</b> 18 depths, up to 30, crossed with four leaf sizes. The best weather
     tree uses depth 3.<br/><br/><b>Forest:</b> four predictor counts crossed with seven node sizes,
    including very shallow trees.',9.5,13,limit=105)
162   r.para(317,579,259,'<b>Ridge / LASSO / elastic net:</b> 61 penalties from 0.001 to 1,000,000 and
    five mixing values.<br/><br/><b>KNN:</b> nine neighbor counts, from 3 to 240. Boundary optima
    triggered wider grids before evaluation.',9.5,13,limit=105)
163   r.note(705,'The January 2024 outcomes played no role in choosing the method, settings or benchmark.
     All models were fitted on the full 2023 development sample, and their future predictions were saved
     before evaluation outcomes were joined.')
164
165   def future(r):
166     r.start('The future test')
167     r.title('04 / JANUARY 2024 · 31 DATES · 93 AIRPORT-DAYS','January breaks the winning
    streak.','Morning weather does not lower future RMSE. Against the airport average, MAE falls from
    15.12 to 13.98 minutes, but the largest delays go largely unanticipated. Three dates account for
    69.5% of its squared error.')
168     # Three aligned time series expose the correlated misses, rather than hiding them in a score.
169     x0,x1=73,400;xx=lambda day:x0+(day-1)/30*(x1-x0)
170     for j,ap in enumerate(['EWR','JFK','LGA']):
171       top=197+j*112;bottom=top+78;yy=lambda v:bottom-v/130*78
172       r.text(36,top-18,ap,11,'Bold')
173       for t in [0,60,120]:r.line(x0,yy(t),x1,yy(t));r.text(x0-8,yy(t)-3,str(t),7,'Sans',MUTED,'right')
174       z=sorted([v for v in EV if v['origin']==ap],key=lambda v:v['date'])
175       for key,c in [('delay',INK),('Morning weather',ORANGE)]:
176         for a,b in zip(z,z[1:]):r.line(xx(int(a['date'][-2:])),yy(val(a,key)),xx(int(b['date'][-
    2:])),yy(val(b,key)),c,1.5)
177       if ap=='LGA':
178         for day in [16,19]:
179           a=z[day-1];r.dot(xx(day),yy(val(a,'delay')),3,INK)
180         r.text(430,top+2,'JAN 16 / LGA',8,'Bold',ORANGE)
181         r.text(430,top+22,'120 min',21,'Bold')
182         r.text(430,top+51,'Predicted: 20 min',8,'Sans',MUTED)
183       if ap=='EWR':
184         r.text(430,top,'DAILY MEAN DELAY',7,'Bold',MUTED)
185         r.line(430,top+28,450,top+28,INK,1.8);r.text(458,top+24,'Observed',8)
186         r.line(430,top+49,450,top+49,ORANGE,1.8);r.text(458,top+45,'9 a.m. forecast',8)
187       if ap=='JFK':
188         r.para(430,top,146,'The same difficult dates hit multiple airports. Uncertainty must keep those
    shared shocks together.',9,12,limit=80)
189     for day in [1,8,16,24,31]:r.text(xx(day),510,f'Jan {day}',7.5,'Sans',MUTED,'center')
190     r.line(36,540,576,540,INK,1)
191     for x,n,l in [(36,'20.76','AIRPORT-MEAN RMSE'),(221,'20.89','WEATHER-FOREST
    RMSE'),(406,'13.98','WEATHER-FOREST MAE')]:
192       r.text(x,558,n,29,'Bold',ORANGE if x==221 else INK);r.text(x,598,l,7.3,'Bold',MUTED)
193     r.para(36,629,259,'<b>No demonstrated RMSE gain.</b> The change is <b>-0.14 minutes</b> of
    improvement. A paired seven-day block bootstrap gives a 95% interval of <b>-1.44 to +1.70
    minutes</b>. The fixed-family forest comparison also straddles zero.',9.5,13,limit=90)
194     r.para(317,629,259,'<b>Decision:</b> do not replace the simple benchmark on this evidence. Test
```

report.py

Part 5

```
    more seasons and add forecasts of afternoon weather and network disruption. January has only 31
    dates; its 1,025 missing departure outcomes are another blind spot.',9.5,13,limit=90)
195  r.note(719,'Intervals use 2,000 resamples of seven-day blocks, keeping all airports together and
    the fitted models fixed. RMSE is in minutes of airport-day mean delay, not individual passenger
    delay. Lower is better.')
196
197 def appendix(r):
198  for name in ['PROTOCOL.md','download.py','prepare.R','methods.R','select.R','lock.R','evaluate.R','
    describe.R','finish.R','example.Rmd','render.R','reproduce.R','report.py']:
199   expanded=[]
200   for no,line in enumerate((ROOT/name).read_text().splitlines(),1):
201    for k,s in enumerate(textwrap.wrap(line,100,replace_whitespace=False,drop_whitespace=False) or
    ['']):expanded.append((f'{no:4}  ' if k==0 else '      ')+s)
202   for offset in range(0,len(expanded),68):
203    r.start('Complete source');r.text(36,65,name,18,'Bold');r.text(576,72,f'Part
    {offset//68+1}',8,'Sans',MUTED,'right')
204    for j,line in enumerate(expanded[offset:offset+68]):r.text(36,103+j*9.3,line,7.8,'Mono')
205  r.start('Evidence + provenance')
206  r.title('REPRODUCIBILITY','Open the whole analysis.','The archive includes raw source snapshots,
    complete grids, paired validation predictions, frozen future predictions, evaluation results and
    audit checks.')
207  y=187
208  for title,url in [('2023 flights /
    nycflights23','https://moderndive.github.io/nycflights23/'),('2024 flights / BTS monthly
    records','https://www.transtats.bts.gov/PREZIP/'),('Hourly weather /
    IEM','https://mesonet.agron.iastate.edu/request/download.phtml'),('Timestamp construction / upstream
     source','https://github.com/simonpcouch/anyflights/blob/main/R/utils.R')]:
209   r.text(36,y,title,11,'Bold',BLUE);r.text(36,y+20,url,8,'Sans',MUTED);r.c.linkURL(url,(36,792-y-
    36,576,792-y),relative=0);y+=65
210  r.para(36,477,540,'<b>Time handling matters.</b> The upstream flight package copies local scheduled
     hours into a UTC-tagged timestamp without conversion. We use the original local calendar fields and
     scheduled HHMM values instead. Weather for both years was retrieved with explicit America/New_York
    output. The source code and cutoff checks make this choice inspectable.',10,14,limit=95)
211  r.para(36,588,540,'<b>Render:</b> run Rscript render.R. <b>Rebuild:</b> run Rscript reproduce.R.
    Both report editions come from the same parameterized R Markdown source. The code edition appends
    every analysis and layout script. Rerunning a saved evaluation reproduces it; it does not make it
    unopened again. SHA256SUMS verifies the delivered files.',10,14,limit=95)
212
213 if __name__=='__main__':
214  p=argparse.ArgumentParser();p.add_argument('--output',required=True);p.add_argument('--
    code',action='store_true');args=p.parse_args()
215  r=Report(args.output)
216  for page in [opening,patterns,validation,tuning,future]:page(r)
217  if args.code:appendix(r)
218  r.save()
```

REPRODUCIBILITY

Open the whole analysis.

The archive includes raw source snapshots, complete grids, paired validation predictions, frozen future predictions, evaluation results and audit checks.

2023 flights / nycflights23

https://moderndive.github.io/nycflights23/

2024 flights / BTS monthly records

https://www.transtats.bts.gov/PREZIP/

Hourly weather / IEM

https://mesonet.agron.iastate.edu/request/download.phtml

Timestamp construction / upstream source

https://github.com/simonpcouch/anyflights/blob/main/R/utils.R

Time handling matters. The upstream flight package copies local scheduled hours into a UTC-tagged timestamp without conversion. We use the original local calendar fields and scheduled HHMM values instead. Weather for both years was retrieved with explicit America/New_York output. The source code and cutoff checks make this choice inspectable.

Render: run Rscript render.R. **Rebuild:** run Rscript reproduce.R. Both report editions come from the same parameterized R Markdown source. The code edition appends every analysis and layout script. Rerunning a saved evaluation reproduces it; it does not make it unopened again. SHA256SUMS verifies the delivered files.